



**Адаптивные методы. Методы матричной  
оптимизации на основе ортогонализации.  
Shampoo, Muon.**

**Даня Меркулов**

ФКН ВШЭ

## Адаптивные стохастические методы

# Почему нужна адаптивность

- Единый скаляр  $\alpha$  — плохой выбор, когда **разные координаты** имеют существенно разные масштабы:

# Почему нужна адаптивность

- Единый скаляр  $\alpha$  — плохой выбор, когда **разные координаты** имеют существенно разные масштабы:
  - редкие признаки в логистической регрессии и NLP-моделях;

# Почему нужна адаптивность

- Единый скаляр  $\alpha$  — плохой выбор, когда **разные координаты** имеют существенно разные масштабы:
  - редкие признаки в логистической регрессии и NLP-моделях;
  - в embedding-слое строка редкого слова получает ненулевой градиент только когда это слово попало в батч — в отличие от плотных слоёв, где все параметры обновляются на каждом шаге.

# Почему нужна адаптивность

- Единый скаляр  $\alpha$  — плохой выбор, когда **разные координаты** имеют существенно разные масштабы:
  - редкие признаки в логистической регрессии и NLP-моделях;
  - в embedding-слое строка редкого слова получает ненулевой градиент только когда это слово попало в батч — в отличие от плотных слоёв, где все параметры обновляются на каждом шаге.
- Идея адаптивных методов — **подобрать свой эффективный шаг каждой координате**, опираясь на накопленную историю градиентов.

# Почему нужна адаптивность

- Единый скаляр  $\alpha$  — плохой выбор, когда **разные координаты** имеют существенно разные масштабы:
  - редкие признаки в логистической регрессии и NLP-моделях;
  - в embedding-слое строка редкого слова получает ненулевой градиент только когда это слово попало в батч — в отличие от плотных слоёв, где все параметры обновляются на каждом шаге.
- Идея адаптивных методов — **подобрать свой эффективный шаг каждой координате**, опираясь на накопленную историю градиентов.
- Содержательно это можно трактовать как грубое приближение диагонали гессиана (или информационной матрицы Фишера).

# Adagrad <sup>1</sup>

Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ . Обновление для каждой координаты  $j = 1, \dots, p$ :

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

## Замечания:

- Не требует тонкой настройки  $\alpha$  — эффективный шаг убывает автоматически по каждой координате.



# Adagrad <sup>1</sup>

Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ . Обновление для каждой координаты  $j = 1, \dots, p$ :

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

## Замечания:

- Не требует тонкой настройки  $\alpha$  — эффективный шаг убывает автоматически по каждой координате.
- Для редких, но информативных признаков шаг убывает медленно — алгоритм даёт им «работать» дольше.

# Adagrad <sup>1</sup>

Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ . Обновление для каждой координаты  $j = 1, \dots, p$ :

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

## Замечания:

- Не требует тонкой настройки  $\alpha$  — эффективный шаг убывает автоматически по каждой координате.
- Для редких, но информативных признаков шаг убывает медленно — алгоритм даёт им «работать» дольше.
- Заметно улучшает SGD в разреженных задачах (NLP, рекомендательные системы).

# Adagrad <sup>1</sup>

Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ . Обновление для каждой координаты  $j = 1, \dots, p$ :

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

## Замечания:

- Не требует тонкой настройки  $\alpha$  — эффективный шаг убывает автоматически по каждой координате.
- Для редких, но информативных признаков шаг убывает медленно — алгоритм даёт им «работать» дольше.
- Заметно улучшает SGD в разреженных задачах (NLP, рекомендательные системы).
- Главная слабость — **монотонное** накопление в знаменателе: эффективный шаг  $\rightarrow 0$  слишком рано, и обучение «затухает».

---

<sup>1</sup>  Duchi, Hazan, Singer (2010). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.

Решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma) \left(g_j^{(k)}\right)^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

### Замечания:

- В знаменателе — скользящее среднее квадратов градиентов (RMS): «забывает» старые градиенты, в отличие от монотонного накопления в AdaGrad.

Решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma) \left(g_j^{(k)}\right)^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

### Замечания:

- В знаменателе — скользящее среднее квадратов градиентов (RMS): «забывает» старые градиенты, в отличие от монотонного накопления в AdaGrad.
- Эффективный шаг может как убывать, так и возрасть — подходит для нестационарных задач.

Решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma) (g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \varepsilon}}$$

### Замечания:

- В знаменателе — скользящее среднее квадратов градиентов (RMS): «забывает» старые градиенты, в отличие от монотонного накопления в AdaGrad.
- Эффективный шаг может как убывать, так и возрасть — подходит для нестационарных задач.
- Долгое время был выбором по умолчанию при обучении рекуррентных сетей.

---

<sup>2</sup>  Tieleman, Hinton (2012). Coursera Lecture 6.

Развитие RMSProp: вообще **не требует** глобального шага  $\alpha$ . Вместо него масштабирует шаг через накопленные обновления параметров:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma) (g_j^{(k)})^2$$

$$\tilde{g}_j^{(k)} = \frac{\sqrt{\Delta x_j^{(k-1)} + \varepsilon}}{\sqrt{v_j^{(k)} + \varepsilon}} g_j^{(k)}$$

$$x_j^{(k)} = x_j^{(k-1)} - \tilde{g}_j^{(k)}, \quad \Delta x_j^{(k)} = \rho \Delta x_j^{(k-1)} + (1 - \rho) (\tilde{g}_j^{(k)})^2$$

### Замечания:

- Числитель  $\sqrt{\Delta x_j^{(k-1)} + \varepsilon}$  имеет те же единицы, что и параметр — шаг автоматически согласован по масштабу.

Развитие RMSProp: вообще **не требует** глобального шага  $\alpha$ . Вместо него масштабирует шаг через накопленные обновления параметров:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma) (g_j^{(k)})^2$$

$$\tilde{g}_j^{(k)} = \frac{\sqrt{\Delta x_j^{(k-1)} + \varepsilon}}{\sqrt{v_j^{(k)} + \varepsilon}} g_j^{(k)}$$

$$x_j^{(k)} = x_j^{(k-1)} - \tilde{g}_j^{(k)}, \quad \Delta x_j^{(k)} = \rho \Delta x_j^{(k-1)} + (1 - \rho) (\tilde{g}_j^{(k)})^2$$

### Замечания:

- Числитель  $\sqrt{\Delta x_j^{(k-1)} + \varepsilon}$  имеет те же единицы, что и параметр — шаг автоматически согласован по масштабу.
- Полезен, когда масштабы параметров между слоями существенно различаются.

<sup>3</sup>  Zeiler (2012). ADADELTA: An Adaptive Learning Rate Method.



Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов:

EMA: 
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция: 
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление: 
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \varepsilon}$$

### Замечания:

- Корректирует смещение моментов к нулю на старте.

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов:

EMA: 
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция: 
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление: 
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \varepsilon}$$

### Замечания:

- Корректирует смещение моментов к нулю на старте.
- Одна из самых цитируемых работ в области ML.

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов:

EMA: 
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция: 
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление: 
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \varepsilon}$$

### Замечания:

- Корректирует смещение моментов к нулю на старте.
- Одна из самых цитируемых работ в области ML.
- Не сходится для некоторых простых задач (даже выпуклых).

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов:

EMA: 
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция: 
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление: 
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \varepsilon}$$

### Замечания:

- Корректирует смещение моментов к нулю на старте.
- Одна из самых цитируемых работ в области ML.
- Не сходится для некоторых простых задач (даже выпуклых).
- Гораздо лучше работает для языковых моделей, чем для задач компьютерного зрения — открытый вопрос «почему».

<sup>4</sup>  Kingma, Ba (2014). Adam: A Method for Stochastic Optimization.

<sup>5</sup>  Reddi, Kale, Kumar (2018). On the Convergence of Adam and Beyond.

Решает проблему  $\ell_2$ -регуляризации в адаптивных оптимизаторах. При стандартном подходе  $\ell_2$ -регуляризация добавляет  $\lambda\|x\|^2$  к функции потерь, порождая градиентный член  $\lambda x$ . В Adam этот член делится на  $(\sqrt{\hat{v}_j} + \varepsilon)$  — сила регуляризации оказывается зависимой от магнитуд градиентов, что нежелательно.

AdamW **разделяет** weight decay и адаптацию градиентов:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \left( \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \varepsilon} + \lambda x_j^{(k-1)} \right)$$

### Замечания:

- Член weight decay  $\lambda x_j^{(k-1)}$  добавляется *после* адаптивного градиентного шага и не масштабируется на  $\sqrt{\hat{v}_j}$ .

Решает проблему  $\ell_2$ -регуляризации в адаптивных оптимизаторах. При стандартном подходе  $\ell_2$ -регуляризация добавляет  $\lambda\|x\|^2$  к функции потерь, порождая градиентный член  $\lambda x$ . В Adam этот член делится на  $(\sqrt{\hat{v}_j} + \epsilon)$  — сила регуляризации оказывается зависимой от магнитуд градиентов, что нежелательно.

AdamW **разделяет** weight decay и адаптацию градиентов:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \left( \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)$$

## Замечания:

- Член weight decay  $\lambda x_j^{(k-1)}$  добавляется *после* адаптивного градиентного шага и не масштабируется на  $\sqrt{\hat{v}_j}$ .
- Широко используется при обучении трансформеров и больших моделей; выбор по умолчанию в HuggingFace Trainer.

---

<sup>6</sup>  Loshchilov, Hutter (2017). Decoupled Weight Decay Regularization.

## Adam не сходится: постановка <sup>7</sup>

Рассмотрим **простейшую** возможную задачу — выпуклую, гладкую, сильно выпуклую, с единственным глобальным минимумом в нуле:

$$L(w) = w^2, \quad \nabla L(w) = 2w, \quad w \in \mathbb{R}.$$

Обновление Adam (одномерный случай, без коррекции смещения и для упрощения  $\varepsilon = 0$ ):

$$\begin{aligned} g_t &= 2w_{t-1}, \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ w_t &= w_{t-1} - \eta \frac{m_t}{\sqrt{v_t}}. \end{aligned}$$

---

<sup>7</sup>  Reddi et al. (2018). On the Convergence of Adam and Beyond.

## Adam не сходится: постановка <sup>7</sup>

Рассмотрим **простейшую** возможную задачу — выпуклую, гладкую, сильно выпуклую, с единственным глобальным минимумом в нуле:

$$L(w) = w^2, \quad \nabla L(w) = 2w, \quad w \in \mathbb{R}.$$

Обновление Adam (одномерный случай, без коррекции смещения и для упрощения  $\varepsilon = 0$ ):

$$\begin{aligned} g_t &= 2w_{t-1}, \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ w_t &= w_{t-1} - \eta \frac{m_t}{\sqrt{v_t}}. \end{aligned}$$

Чего мы **ждём** от корректного метода:  $w_t \rightarrow 0$  и длина шага  $|w_t - w_{t-1}| \rightarrow 0$  по мере приближения к оптимуму. У GD с  $\eta < 1/2$  это так:  $w_t = (1 - 2\eta)w_{t-1}$ , экспоненциальная сходимость.

Покажем, что у Adam **этого не происходит**.

---

<sup>7</sup>  Reddi et al. (2018). On the Convergence of Adam and Beyond.



## Adam не сходится: анализ масштаба при малых $|w|$

Предположим, что итерации находятся в окрестности нуля и величина  $|w_t|$  ведёт себя как масштаб  $|w| \rightarrow 0$ .  
Что происходит с  $m_t$  и  $v_t$ ?

## Adam не сходится: анализ масштаба при малых $|w|$

Предположим, что итерации находятся в окрестности нуля и величина  $|w_t|$  ведёт себя как масштаб  $|w| \rightarrow 0$ .  
Что происходит с  $m_t$  и  $v_t$ ?

$m_t$  — **линеен по  $w$** . Это сумма (взвешенная) прошлых градиентов  $g_\tau = 2w_{\tau-1}$ , каждый из которых линеен по  $w$ :

$$m_t = (1 - \beta_1) \sum_{\tau \leq t} \beta_1^{t-\tau} 2w_{\tau-1} \propto w.$$

## Adam не сходится: анализ масштаба при малых $|w|$

Предположим, что итерации находятся в окрестности нуля и величина  $|w_t|$  ведёт себя как масштаб  $|w| \rightarrow 0$ .  
Что происходит с  $m_t$  и  $v_t$ ?

$m_t$  — **линеен по  $w$** . Это сумма (взвешенная) прошлых градиентов  $g_\tau = 2w_{\tau-1}$ , каждый из которых линеен по  $w$ :

$$m_t = (1 - \beta_1) \sum_{\tau \leq t} \beta_1^{t-\tau} 2w_{\tau-1} \propto w.$$

$v_t$  — **квадратичен по  $w$** . Сумма прошлых  $g_\tau^2 = 4w_{\tau-1}^2$ , каждый член квадратичен:

$$v_t = (1 - \beta_2) \sum_{\tau \leq t} \beta_2^{t-\tau} 4w_{\tau-1}^2 \propto w^2.$$

## Adam не сходится: анализ масштаба при малых $|w|$

Предположим, что итерации находятся в окрестности нуля и величина  $|w_t|$  ведёт себя как масштаб  $|w| \rightarrow 0$ .  
Что происходит с  $m_t$  и  $v_t$ ?

$m_t$  — **линеен по  $w$** . Это сумма (взвешенная) прошлых градиентов  $g_\tau = 2w_{\tau-1}$ , каждый из которых линеен по  $w$ :

$$m_t = (1 - \beta_1) \sum_{\tau \leq t} \beta_1^{t-\tau} 2w_{\tau-1} \propto w.$$

$v_t$  — **квадратичен по  $w$** . Сумма прошлых  $g_\tau^2 = 4w_{\tau-1}^2$ , каждый член квадратичен:

$$v_t = (1 - \beta_2) \sum_{\tau \leq t} \beta_2^{t-\tau} 4w_{\tau-1}^2 \propto w^2.$$

Следовательно:

$$\sqrt{v_t} \propto |w| \quad \Rightarrow \quad \frac{m_t}{\sqrt{v_t}} \approx \text{sign}(w) = \pm 1$$

Это отношение **не зависит** от  $|w|$ : какой бы маленькой ни стала итерация, нормированный градиент остаётся порядка единицы.

## Adam не сходится: предельный цикл

Из предыдущего шага: шаг Adam в окрестности нуля имеет постоянную **по модулю** длину

$$|w_t - w_{t-1}| = \eta \left| \frac{m_t}{\sqrt{v_t}} \right| \approx \eta,$$

независимо от того, насколько  $w$  близко к оптимуму.

## Adam не сходится: предельный цикл

Из предыдущего шага: шаг Adam в окрестности нуля имеет постоянную **по модулю** длину

$$|w_t - w_{t-1}| = \eta \left| \frac{m_t}{\sqrt{v_t}} \right| \approx \eta,$$

независимо от того, насколько  $w$  близко к оптимуму.

**Что это означает.** Если  $|w_{t-1}| < \eta/2$ , то шаг  $\pm\eta$  «перепрыгивает» через ноль и оказывается с противоположной стороны на расстоянии  $\approx \eta/2$ . На следующем шаге — обратно.

## Adam не сходится: предельный цикл

Из предыдущего шага: шаг Adam в окрестности нуля имеет постоянную **по модулю** длину

$$|w_t - w_{t-1}| = \eta \left| \frac{m_t}{\sqrt{v_t}} \right| \approx \eta,$$

независимо от того, насколько  $w$  близко к оптимуму.

**Что это означает.** Если  $|w_{t-1}| < \eta/2$ , то шаг  $\pm\eta$  «перепрыгивает» через ноль и оказывается с противоположной стороны на расстоянии  $\approx \eta/2$ . На следующем шаге — обратно.

Получаем **предельный цикл**:

$$w_t \in \{-\eta/2, +\eta/2\}, \quad t \rightarrow \infty.$$

Метод **не сходится** ни к точке, ни даже к нулю по среднему.

## Adam не сходится: предельный цикл

Из предыдущего шага: шаг Adam в окрестности нуля имеет постоянную **по модулю** длину

$$|w_t - w_{t-1}| = \eta \left| \frac{m_t}{\sqrt{v_t}} \right| \approx \eta,$$

независимо от того, насколько  $w$  близко к оптимуму.

**Что это означает.** Если  $|w_{t-1}| < \eta/2$ , то шаг  $\pm\eta$  «перепрыгивает» через ноль и оказывается с противоположной стороны на расстоянии  $\approx \eta/2$ . На следующем шаге — обратно.

Получаем **предельный цикл**:

$$w_t \in \{-\eta/2, +\eta/2\}, \quad t \rightarrow \infty.$$

Метод **не сходится** ни к точке, ни даже к нулю по среднему.

**Сравнение с SGD.** Для  $L(w) = w^2$  обновление SGD:

$$w_t = w_{t-1} - \eta \cdot 2w_{t-1} = (1 - 2\eta)w_{t-1}.$$

Шаг **пропорционален**  $w$ , обнуляется в пределе  $\Rightarrow$  сходимость.



## Adam не сходится: иллюстрация

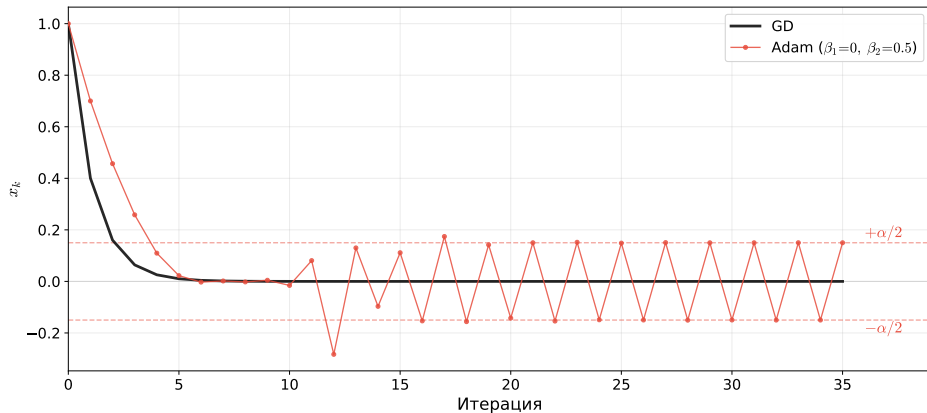
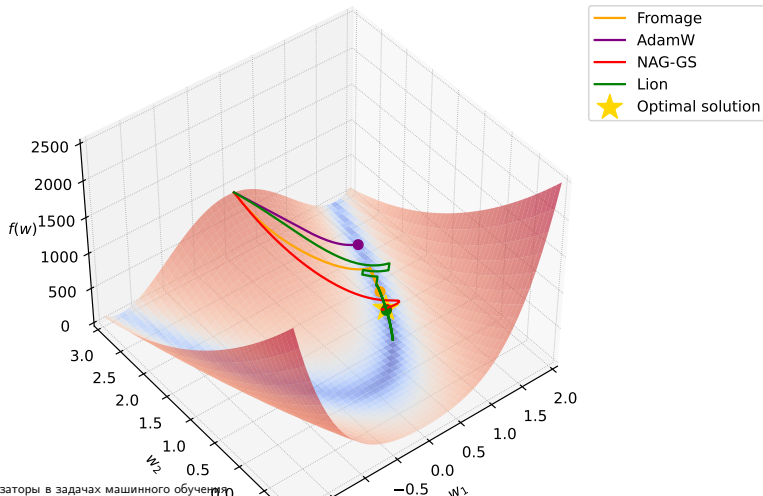


Figure 1: GD сходится к нулю, Adam выходит на предельный цикл амплитуды  $\sim \alpha/2$ .

## Как сравнивать оптимизаторы в задачах машинного обучения

# Много оптимизаторов — как сравнивать?

Rosenbrock Function.  
Adaptive stochastic gradient algorithms.  
Learning rate 0.003



## AlgoPerf benchmark <sup>8</sup>

- **AlgoPerf** — стандартизированный бенчмарк для сравнения алгоритмов обучения нейросетей по двум регламентам:

## AlgoPerf benchmark <sup>8</sup>

- **AlgoPerf** — стандартизированный бенчмарк для сравнения алгоритмов обучения нейросетей по двум регламентам:
  - **External Tuning** — подбор гиперпараметров с ограниченными ресурсами (5 попыток);

## AlgoPerf benchmark <sup>8</sup>

- **AlgoPerf** — стандартизированный бенчмарк для сравнения алгоритмов обучения нейросетей по двум регламентам:
  - **External Tuning** — подбор гиперпараметров с ограниченными ресурсами (5 попыток);
  - **Self-Tuning** — автоматическая настройка на одной машине (3× бюджет).

- **AlgoPerf** — стандартизированный бенчмарк для сравнения алгоритмов обучения нейросетей по двум регламентам:
  - **External Tuning** — подбор гиперпараметров с ограниченными ресурсами (5 попыток);
  - **Self-Tuning** — автоматическая настройка на одной машине (3× бюджет).
- **Оценка** агрегируется через профили производительности; финальный балл — нормализованная площадь под профилем (1.0 = быстрееший на всех задачах).

# AlgoPerf benchmark <sup>8</sup>

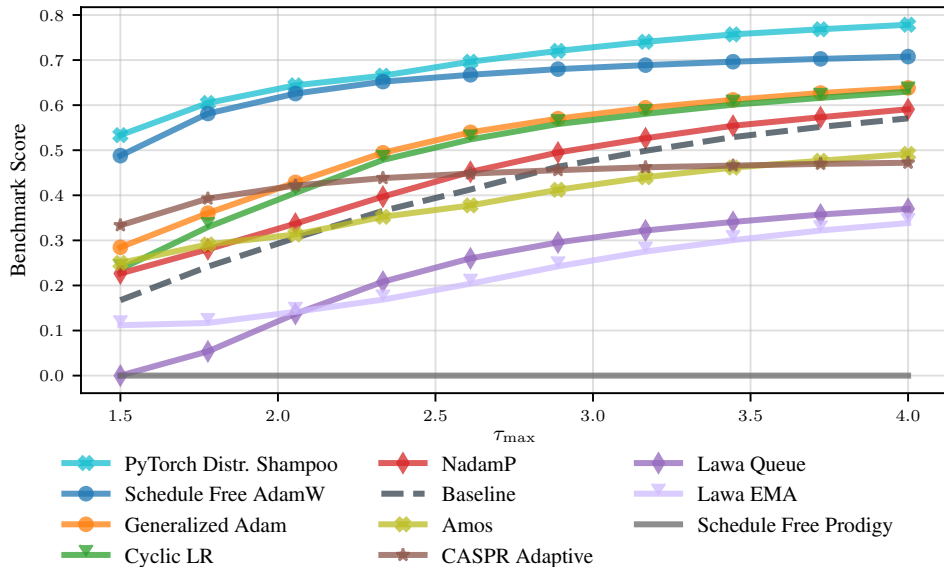
- **AlgoPerf** — стандартизированный бенчмарк для сравнения алгоритмов обучения нейросетей по двум регламентам:
  - **External Tuning** — подбор гиперпараметров с ограниченными ресурсами (5 попыток);
  - **Self-Tuning** — автоматическая настройка на одной машине (3× бюджет).
- **Оценка** агрегируется через профили производительности; финальный балл — нормализованная площадь под профилем (1.0 = быстрееший на всех задачах).
- **Стоимость** оценки: ~49 240 часов работы на 8 NVIDIA V100 GPU.

---

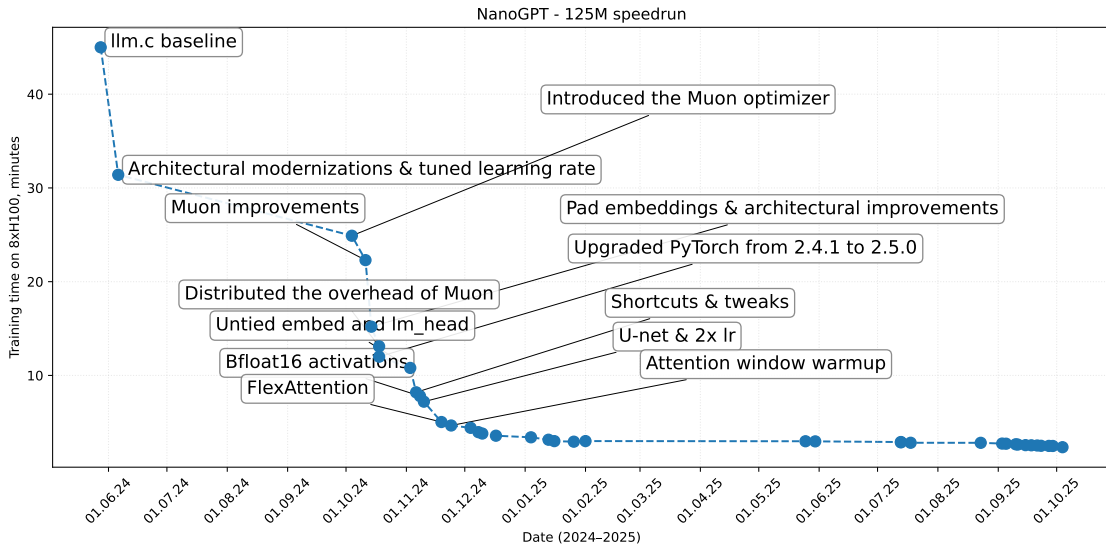
<sup>8</sup>  Dahl et al. (2023). Benchmarking Neural Network Training Algorithms.



## AlgoPerf benchmark: результаты



# NanoGPT speedrun <sup>9</sup>



<sup>9</sup> Источник

## Работают ли трюки, если увеличить размер модели?

Scaling up the NanoGPT (124M) speedrun

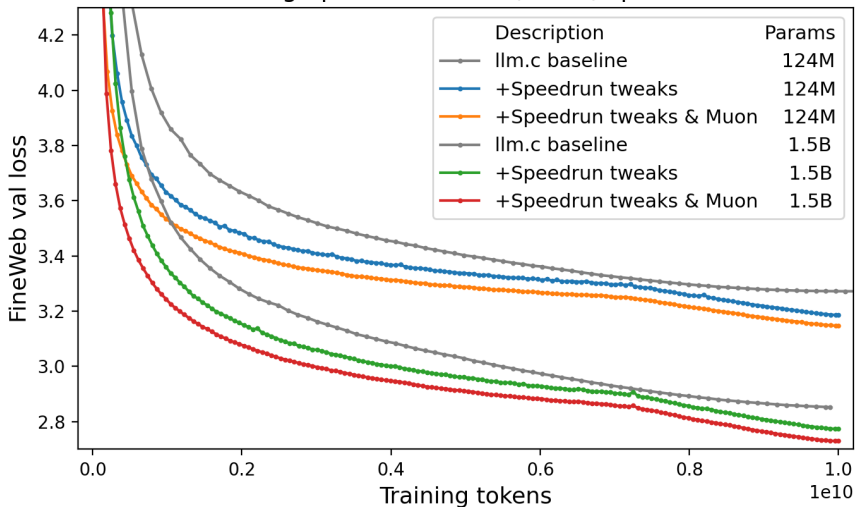


Figure 2:  Источник

# Сравнение оптимизаторов на NanoGPT speedrun

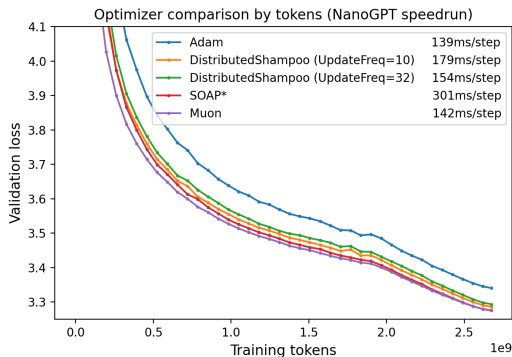


Figure 3: По числу токенов

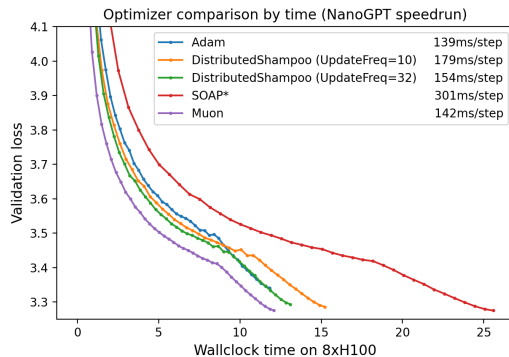
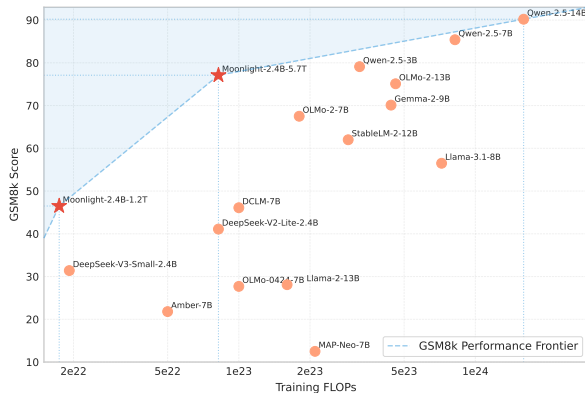
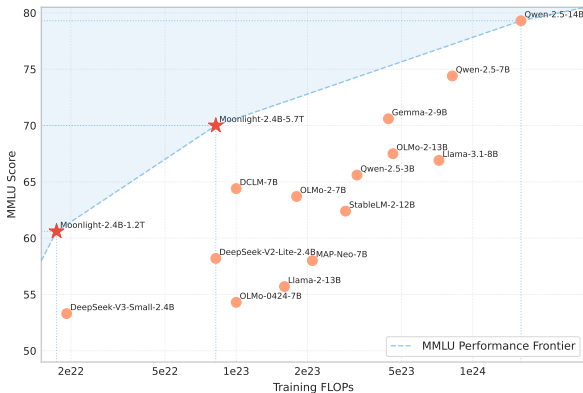


Figure 4: По wallclock time ( $8 \times H100$ )

Муон (фиолетовый) — лучший по обоим метрикам: быстрая сходимость по токенам и самый дешёвый по времени (142 ms/step).

Muon

# Новый подход к оптимизации <sup>10</sup>



Модели, отмеченные звёздочкой, были обучены методом Muon, остальные модели были обучены другими алгоритмами оптимизации.

$$\min_{x \in \mathbb{R}^p} f(x)$$

$$f(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \mathcal{O}(\|x - x_k\|_2^2).$$

Функция потерь

$$\min_{x \in \mathbb{R}^p} f(x)$$


$$f(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \mathcal{O}(\|x - x_k\|_2^2).$$



$$\min_{x \in \mathbb{R}^p} f(x)$$

Функция потерь

$$f(x) = \underbrace{f(x_k) + \langle \nabla f(x_k), x - x_k \rangle}_{\text{Линейная аппроксимация}} + \mathcal{O}(\|x - x_k\|_2^2).$$

Линейная  
аппроксимация

Функция потерь

$$\min_{x \in \mathbb{R}^p} f(x)$$

$$f(x) = \underbrace{f(x_k) + \langle \nabla f(x_k), x - x_k \rangle}_{\text{Линейная аппроксимация}} + \mathcal{O}(\|x - x_k\|_2^2).$$

Линейная  
аппроксимация

Хорошее приближение  
в окрестности  $x_k$

## Интуиция за методом Muon. Градиентный спуск

$$x_{k+1} = \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right)$$

## Интуиция за методом Муон. Градиентный спуск

$$\begin{aligned}x_{k+1} &= \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right) \\ &= x_k - \alpha \nabla f(x_k)\end{aligned}$$

## Интуиция за методом Муон. Градиентный спуск

Штраф за  
дальность от  $x_k$

$$\begin{aligned}x_{k+1} &= \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right) \\ &= x_k - \alpha \nabla f(x_k)\end{aligned}$$

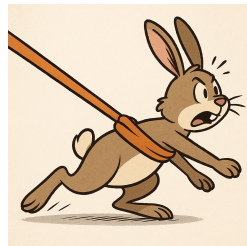
## Интуиция за методом Muon. Градиентный спуск

$$x_{k+1} = \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right)$$

$$= x_k - \alpha \nabla f(x_k)$$

Шаг обучения /  
коэффициент регуляризации

Штраф за  
дальность от  $x_k$



## Интуиция за методом Muon. Нормированный градиентный спуск

$$x_{k+1} = \operatorname{argmin}_{\|x - x_k\|_2 = \alpha} (f(x_k) + \langle \nabla f(x_k), x - x_k \rangle)$$

## Интуиция за методом Муон. Нормированный градиентный спуск

$$\begin{aligned}x_{k+1} &= \operatorname{argmin}_{\|x - x_k\|_2 = \alpha} (f(x_k) + \langle \nabla f(x_k), x - x_k \rangle) \\&= x_k - \alpha \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2}\end{aligned}$$



## Интуиция за методом Муон. Нормированный градиентный спуск

$$\begin{aligned}x_{k+1} &= \operatorname{argmin}_{\|x - x_k\|_2 = \alpha} (f(x_k) + \langle \nabla f(x_k), x - x_k \rangle) \\ &= x_k - \alpha \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2}\end{aligned}$$

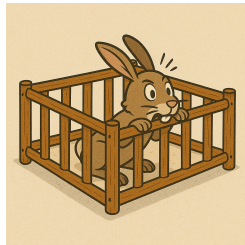

Ограничение на длину шага

## Интуиция за методом Muon. Нормированный градиентный спуск

$$\begin{aligned}x_{k+1} &= \operatorname{argmin}_{\|x - x_k\|_2 = \alpha} (f(x_k) + \langle \nabla f(x_k), x - x_k \rangle) \\&= x_k - \alpha \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2}\end{aligned}$$

Ограничение на длину шага

Параметр ограничения / шаг обучения



## Что насчёт других норм?

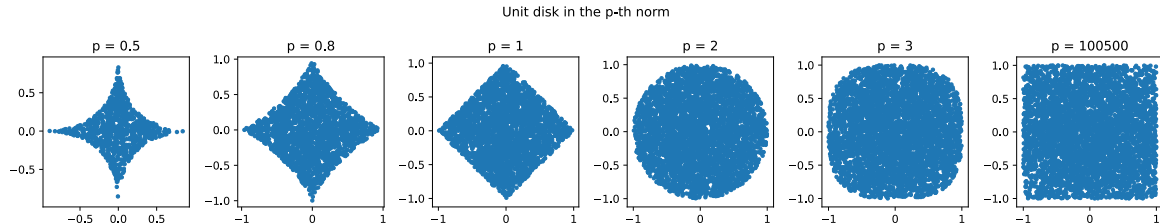


Figure 5: Примеры шаров в разных нормах

## Для неевклидовых норм нужно ввести несколько определений

- Сопряжённая норма:

$$\|g\|^* = \sup_{\|x\|=1} \langle g, x \rangle$$

## Для неевклидовых норм нужно ввести несколько определений

- Сопряжённая норма:

$$\|g\|^* = \sup_{\|x\|=1} \langle g, x \rangle$$

- Linear Minimization Oracle:

$$\text{LMO}_{\|\cdot\|}(g) = \underset{\|x\|=1}{\operatorname{argmin}} \langle g, x \rangle$$

## Для неевклидовых норм нужно ввести несколько определений

- Сопряжённая норма:

$$\|g\|^* = \sup_{\|x\|=1} \langle g, x \rangle$$

- Linear Minimization Oracle:

$$\text{LMO}_{\|\cdot\|}(g) = \underset{\|x\|=1}{\operatorname{argmin}} \langle g, x \rangle$$

- Важное свойство, связывающее эти два понятия:

$$\langle g, \text{LMO}_{\|\cdot\|}(g) \rangle = -\|g\|^*$$

## ! Неевклидов градиентный спуск

Для вектора градиента  $g = \nabla f(x_k)$  и шага  $\alpha > 0$ :

$$x_{k+1} = \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle g, x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|^2 \right) = x_k + \alpha \|g\|^* \operatorname{LMO}_{\|\cdot\|}(g)$$

### ! Неевклидов градиентный спуск

Для вектора градиента  $g = \nabla f(x_k)$  и шага  $\alpha > 0$ :

$$x_{k+1} = \operatorname{argmin}_{x \in \mathbb{R}^p} \left( f(x_k) + \langle g, x - x_k \rangle + \frac{1}{2\alpha} \|x - x_k\|^2 \right) = x_k + \alpha \|g\|^* \operatorname{LMO}_{\|\cdot\|}(g)$$

### ! Неевклидов нормированный градиентный спуск

Для вектора градиента  $g = \nabla f(x_k)$  и шага  $\alpha > 0$ :

$$x_{k+1} = \operatorname{argmin}_{\|x - x_k\| = \alpha} (f(x_k) + \langle g, x - x_k \rangle) = x_k + \alpha \operatorname{LMO}_{\|\cdot\|}(g)$$



## Вывод: неевклидов градиентный спуск

$$\min_{d \in \mathbb{R}^p} \left( \langle g, d \rangle + \frac{1}{2\alpha} \|d\|^2 \right)$$

## Вывод: неевклидов градиентный спуск

$$\min_{d \in \mathbb{R}^p} \left( \langle g, d \rangle + \frac{1}{2\alpha} \|d\|^2 \right)$$

Подставим  $d = r \cdot u$ , где  $\|u\| = 1$ ,  $r \geq 0$ :

$$\min_{r \geq 0, \|u\|=1} \left( r \langle g, u \rangle + \frac{r^2}{2\alpha} \right)$$

## Вывод: неевклидов градиентный спуск

$$\min_{d \in \mathbb{R}^p} \left( \langle g, d \rangle + \frac{1}{2\alpha} \|d\|^2 \right)$$

Подставим  $d = r \cdot u$ , где  $\|u\| = 1$ ,  $r \geq 0$ :

$$\min_{r \geq 0, \|u\|=1} \left( r \langle g, u \rangle + \frac{r^2}{2\alpha} \right)$$

При фиксированном  $u$  оптимальное  $r^* = -\alpha \langle g, u \rangle$  (при  $\langle g, u \rangle < 0$ ). Подставляя:

$$-\frac{\alpha}{2} \langle g, u \rangle^2 \rightarrow \min_{\|u\|=1}$$

## Вывод: неевклидов градиентный спуск

$$\min_{d \in \mathbb{R}^p} \left( \langle g, d \rangle + \frac{1}{2\alpha} \|d\|^2 \right)$$

Подставим  $d = r \cdot u$ , где  $\|u\| = 1$ ,  $r \geq 0$ :

$$\min_{r \geq 0, \|u\|=1} \left( r \langle g, u \rangle + \frac{r^2}{2\alpha} \right)$$

При фиксированном  $u$  оптимальное  $r^* = -\alpha \langle g, u \rangle$  (при  $\langle g, u \rangle < 0$ ). Подставляя:

$$-\frac{\alpha}{2} \langle g, u \rangle^2 \rightarrow \min_{\|u\|=1}$$

Максимум  $\langle g, u \rangle^2$  при  $\|u\| = 1$  достигается на  $u^* = \text{LMO}_{\|\cdot\|}(g)$ , где  $\langle g, u^* \rangle = -\|g\|^*$ :

$$d^* = \alpha \|g\|^* \cdot \text{LMO}_{\|\cdot\|}(g) \implies x_{k+1} = x_k + \alpha \|g\|^* \text{LMO}_{\|\cdot\|}(g)$$

## Вывод: неевклидов нормированный градиентный спуск

$$\min_{\|d\|=\alpha} \langle g, d \rangle$$

## Вывод: неевклидов нормированный градиентный спуск

$$\min_{\|d\|=\alpha} \langle g, d \rangle$$

Подставим  $d = \alpha u$ , где  $\|u\| = 1$ :

$$\min_{\|u\|=1} \alpha \langle g, u \rangle = \alpha \cdot \min_{\|u\|=1} \langle g, u \rangle$$

## Вывод: неевклидов нормированный градиентный спуск

$$\min_{\|d\|=\alpha} \langle g, d \rangle$$

Подставим  $d = \alpha u$ , где  $\|u\| = 1$ :

$$\min_{\|u\|=1} \alpha \langle g, u \rangle = \alpha \cdot \min_{\|u\|=1} \langle g, u \rangle$$

По определению  $\text{LMO}_{\|\cdot\|}(g) = \operatorname{argmin}_{\|u\|=1} \langle g, u \rangle$ :

$$d^* = \alpha \cdot \text{LMO}_{\|\cdot\|}(g) \implies x_{k+1} = x_k + \alpha \text{LMO}_{\|\cdot\|}(g)$$

## Вывод: неевклидов нормированный градиентный спуск

$$\min_{\|d\|=\alpha} \langle g, d \rangle$$

Подставим  $d = \alpha u$ , где  $\|u\| = 1$ :

$$\min_{\|u\|=1} \alpha \langle g, u \rangle = \alpha \cdot \min_{\|u\|=1} \langle g, u \rangle$$

По определению  $\text{LMO}_{\|\cdot\|}(g) = \operatorname{argmin}_{\|u\|=1} \langle g, u \rangle$ :

$$d^* = \alpha \cdot \text{LMO}_{\|\cdot\|}(g) \implies x_{k+1} = x_k + \alpha \text{LMO}_{\|\cdot\|}(g)$$

**Сравнение:** градиентный спуск масштабирует шаг на  $\|g\|^*$  (зависит от величины градиента), нормированный — нет (фиксированная длина шага  $\alpha$ ).



## В нейросетях параметры — матрицы

- В линейных слоях, attention, embedding-слоях параметр — матрица весов

$$W \in \mathbb{R}^{d \times n}, \quad G_k = \nabla_W f(W_k) \in \mathbb{R}^{d \times n}.$$

## В нейросетях параметры — матрицы

- В линейных слоях, attention, embedding-слоях параметр — матрица весов

$$W \in \mathbb{R}^{d \times n}, \quad G_k = \nabla_W f(W_k) \in \mathbb{R}^{d \times n}.$$

- Естественно использовать **матричные нормы**: операторную  $\|\cdot\|_{\text{op}}$ , ядерную  $\|\cdot\|_{\text{нuc}}$ , Фробениуса  $\|\cdot\|_F$  и т.п.

## В нейросетях параметры — матрицы

- В линейных слоях, attention, embedding-слоях параметр — матрица весов

$$W \in \mathbb{R}^{d \times n}, \quad G_k = \nabla_W f(W_k) \in \mathbb{R}^{d \times n}.$$

- Естественно использовать **матричные нормы**: операторную  $\|\cdot\|_{\text{op}}$ , ядерную  $\|\cdot\|_{\text{нук}}$ , Фробениуса  $\|\cdot\|_F$  и т.п.
- Вся логика переносится: вместо вектора ищем «лучшее направление спуска» среди матриц заданной нормы.

## В нейросетях параметры — матрицы

- В линейных слоях, attention, embedding-слоях параметр — матрица весов

$$W \in \mathbb{R}^{d \times n}, \quad G_k = \nabla_W f(W_k) \in \mathbb{R}^{d \times n}.$$

- Естественно использовать **матричные нормы**: операторную  $\|\cdot\|_{\text{op}}$ , ядерную  $\|\cdot\|_{\text{нuc}}$ , Фробениуса  $\|\cdot\|_F$  и т.п.
- Вся логика переносится: вместо вектора ищем «лучшее направление спуска» среди матриц заданной нормы.
- Скалярное произведение:

$$\langle A, B \rangle := \text{tr}(A^\top B) = \sum_{ij} A_{ij} B_{ij}.$$

## Неевклидов нормированный спуск для матриц

Пусть заданы матричная норма  $\|\cdot\|$  и шаг  $\lambda > 0$ . Тогда нормированный шаг по матрице  $W$ :

$$W_{k+1} = \operatorname{argmin}_{\|W - W_k\| = \lambda} \left( f(W_k) + \langle G_k, W - W_k \rangle \right)$$

## Неевклидов нормированный спуск для матриц

Пусть заданы матричная норма  $\|\cdot\|$  и шаг  $\lambda > 0$ . Тогда нормированный шаг по матрице  $W$ :

$$W_{k+1} = \operatorname{argmin}_{\|W - W_k\| = \lambda} \left( f(W_k) + \langle G_k, W - W_k \rangle \right)$$

Подставим  $D = W - W_k$ ,  $\|D\| = \lambda$ ,  $D = \lambda U$ ,  $\|U\| = 1$ :

$$\min_{\|U\|=1} \lambda \langle G_k, U \rangle = \lambda \langle G_k, \operatorname{LMO}_{\|\cdot\|}(G_k) \rangle$$

## Неевклидов нормированный спуск для матриц

Пусть заданы матричная норма  $\|\cdot\|$  и шаг  $\lambda > 0$ . Тогда нормированный шаг по матрице  $W$ :

$$W_{k+1} = \operatorname{argmin}_{\|W - W_k\| = \lambda} \left( f(W_k) + \langle G_k, W - W_k \rangle \right)$$

Подставим  $D = W - W_k$ ,  $\|D\| = \lambda$ ,  $D = \lambda U$ ,  $\|U\| = 1$ :

$$\min_{\|U\|=1} \lambda \langle G_k, U \rangle = \lambda \langle G_k, \operatorname{LMO}_{\|\cdot\|}(G_k) \rangle$$

$$W_{k+1} = W_k + \lambda \operatorname{LMO}_{\|\cdot\|}(G_k)$$

## Неевклидов нормированный спуск для матриц

Пусть заданы матричная норма  $\|\cdot\|$  и шаг  $\lambda > 0$ . Тогда нормированный шаг по матрице  $W$ :

$$W_{k+1} = \operatorname{argmin}_{\|W - W_k\| = \lambda} \left( f(W_k) + \langle G_k, W - W_k \rangle \right)$$

Подставим  $D = W - W_k$ ,  $\|D\| = \lambda$ ,  $D = \lambda U$ ,  $\|U\| = 1$ :

$$\min_{\|U\|=1} \lambda \langle G_k, U \rangle = \lambda \langle G_k, \operatorname{LMO}_{\|\cdot\|}(G_k) \rangle$$

$$W_{k+1} = W_k + \lambda \operatorname{LMO}_{\|\cdot\|}(G_k)$$

где

$$\operatorname{LMO}_{\|\cdot\|}(G) = \operatorname{argmin}_{\|W\|=1} \langle G, W \rangle$$

— тот же самый LMO, только теперь он ищет **матрицу** единичной нормы, дающую наибольшее убывание линейного приближения.



# Операторная норма и быстрый расчёт ( $UV^T$ )

Рассмотрим операторную (спектральную) норму  $\|\cdot\|_{\text{op}}$ . Пусть

$$G_k = U\Sigma V^T$$

— редуцированное SVD градиента. Тогда

- LMO по операторной норме:

$$\text{LMO}_{\|\cdot\|}(G) = -UV^T,$$

то есть оптимальное направление — **полярный фактор** матрицы  $G_k$  (ортогональная часть полярного разложения  $G = UP$ ).

# Операторная норма и быстрый расчёт ( $UV^\top$ )

Рассмотрим операторную (спектральную) норму  $\|\cdot\|_{\text{op}}$ . Пусть

$$G_k = U\Sigma V^\top$$

— редуцированное SVD градиента. Тогда

- LMO по операторной норме:

$$\text{LMO}_{\|\cdot\|}(G) = -UV^\top,$$

то есть оптимальное направление — **полярный фактор** матрицы  $G_k$  (ортогональная часть полярного разложения  $G = UP$ ).

- Проблема: полное SVD стоит  $O(d^3)$  на каждом шаге. Но нам нужен только произведение  $UV^\top$ , а не сами  $U$ ,  $\Sigma$ ,  $V$  по отдельности.

# Операторная норма и быстрый расчёт ( $UV^\top$ )

Рассмотрим операторную (спектральную) норму  $\|\cdot\|_{\text{op}}$ . Пусть

$$G_k = U\Sigma V^\top$$

— редуцированное SVD градиента. Тогда

- LMO по операторной норме:

$$\text{LMO}_{\|\cdot\|}(G) = -UV^\top,$$

то есть оптимальное направление — **полярный фактор** матрицы  $G_k$  (ортогональная часть полярного разложения  $G = UP$ ).

- Проблема: полное SVD стоит  $O(d^3)$  на каждом шаге. Но нам нужен только произведение  $UV^\top$ , а не сами  $U$ ,  $\Sigma$ ,  $V$  по отдельности.
- Итерации **Newton–Schulz** используют только матричные умножения и дают приближение  $UV^\top$  за 5–10 шагов — снимают вычислительное узкое место Muon.

## Муон: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

## Муон: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса):  $D^* = -G/\|G\|_F$  — направление  $\propto$  градиенту

## Муон: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса):  $D^* = -G/\|G\|_F$  — направление  $\propto$  градиенту
- **SignGD** ( $\ell_\infty$ -норма):  $D^* = -\text{sign}(G)$  — знак каждого элемента

## Muon: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса):  $D^* = -G/\|G\|_F$  — направление  $\propto$  градиенту
- **SignGD** ( $\ell_\infty$ -норма):  $D^* = -\text{sign}(G)$  — знак каждого элемента
- **Muon** (спектральная норма):  $D^* = -UV^\top$  — полярный фактор градиента

## Муон: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса):  $D^* = -G/\|G\|_F$  — направление  $\propto$  градиенту
- **SignGD** ( $\ell_\infty$ -норма):  $D^* = -\text{sign}(G)$  — знак каждого элемента
- **Муон** (спектральная норма):  $D^* = -UV^\top$  — полярный фактор градиента

---

<sup>13</sup>  J. Bernstein “Deriving Muon”. 2025.

<sup>14</sup>  Kovalev, D. (2025). Understanding Gradient Orthogonalization.



## Муон: вывод через наискорейший спуск в спектральной норме <sup>13 14</sup>

Задача наискорейшего спуска для матричного параметра  $W$ :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где  $\|\cdot\|_\sigma$  — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса):  $D^* = -G/\|G\|_F$  — направление  $\propto$  градиенту
- **SignGD** ( $\ell_\infty$ -норма):  $D^* = -\text{sign}(G)$  — знак каждого элемента
- **Muon** (спектральная норма):  $D^* = -UV^\top$  — полярный фактор градиента

Двойственная норма к спектральной — **ядерная** (сумма сингулярных чисел):

$$\|G\|_* = \sum_i \sigma_i(G), \quad \arg \min_{\|D\|_\sigma \leq 1} \langle G, D \rangle = -UV^\top$$

---

<sup>13</sup>  J. Bernstein “Deriving Muon”. 2025.

<sup>14</sup>  Kovalev, D. (2025). Understanding Gradient Orthogonalization.

## Вывод: SGD как наискорейший спуск в норме Фробениуса

$$D^* = \operatorname{argmin}_{\|D\|_F \leq 1} \langle G, D \rangle$$

## Вывод: SGD как наискорейший спуск в норме Фробениуса

$$D^* = \operatorname{argmin}_{\|D\|_F \leq 1} \langle G, D \rangle$$

Двойственная норма к  $\|\cdot\|_F$  — это  $\|\cdot\|_F$  (норма Фробениуса самосопряжена):

$$\|G\|_F^* = \sup_{\|D\|_F=1} \langle G, D \rangle = \|G\|_F$$

## Вывод: SGD как наискорейший спуск в норме Фробениуса

$$D^* = \operatorname{argmin}_{\|D\|_F \leq 1} \langle G, D \rangle$$

Двойственная норма к  $\|\cdot\|_F$  — это  $\|\cdot\|_F$  (норма Фробениуса самосопряжена):

$$\|G\|_F^* = \sup_{\|D\|_F=1} \langle G, D \rangle = \|G\|_F$$

$\text{LMO}_{\|\cdot\|_F}(G)$ : ищем  $\|D\|_F = 1$ , минимизирующую  $\langle G, D \rangle$ . Решение по Коши–Шварцу:

$$D^* = -\frac{G}{\|G\|_F}$$

## Вывод: SGD как наискорейший спуск в норме Фробениуса

$$D^* = \operatorname{argmin}_{\|D\|_F \leq 1} \langle G, D \rangle$$

Двойственная норма к  $\|\cdot\|_F$  — это  $\|\cdot\|_F$  (норма Фробениуса самосопряжена):

$$\|G\|_F^* = \sup_{\|D\|_F=1} \langle G, D \rangle = \|G\|_F$$

$\operatorname{LMO}_{\|\cdot\|_F}(G)$ : ищем  $\|D\|_F = 1$ , минимизирующую  $\langle G, D \rangle$ . Решение по Коши–Шварцу:

$$D^* = -\frac{G}{\|G\|_F}$$

$$W_{k+1} = W_k - \alpha \frac{G_k}{\|G_k\|_F} = \text{нормированный SGD}$$

## Вывод: SignGD как наискорейший спуск в $\ell_\infty$ -норме

$$D^* = \operatorname{argmin}_{\|D\|_\infty \leq 1} \langle G, D \rangle = \operatorname{argmin}_{\max_{ij} |D_{ij}| \leq 1} \sum_{ij} G_{ij} D_{ij}$$

## Вывод: SignGD как наискорейший спуск в $\ell_\infty$ -норме

$$D^* = \operatorname{argmin}_{\|D\|_\infty \leq 1} \langle G, D \rangle = \operatorname{argmin}_{\max_{ij} |D_{ij}| \leq 1} \sum_{ij} G_{ij} D_{ij}$$

Каждый элемент  $D_{ij}$  независимо ограничен  $|D_{ij}| \leq 1$ . Минимум  $G_{ij} D_{ij}$  достигается при:

$$D_{ij}^* = -\operatorname{sign}(G_{ij})$$

## Вывод: SignGD как наискорейший спуск в $\ell_\infty$ -норме

$$D^* = \operatorname{argmin}_{\|D\|_\infty \leq 1} \langle G, D \rangle = \operatorname{argmin}_{\max_{ij} |D_{ij}| \leq 1} \sum_{ij} G_{ij} D_{ij}$$

Каждый элемент  $D_{ij}$  независимо ограничен  $|D_{ij}| \leq 1$ . Минимум  $G_{ij} D_{ij}$  достигается при:

$$D_{ij}^* = -\operatorname{sign}(G_{ij})$$

Двойственная норма:  $\|G\|_\infty^* = \|G\|_1 = \sum_{ij} |G_{ij}|$



## Вывод: SignGD как наискорейший спуск в $\ell_\infty$ -норме

$$D^* = \operatorname{argmin}_{\|D\|_\infty \leq 1} \langle G, D \rangle = \operatorname{argmin}_{\max_{ij} |D_{ij}| \leq 1} \sum_{ij} G_{ij} D_{ij}$$

Каждый элемент  $D_{ij}$  независимо ограничен  $|D_{ij}| \leq 1$ . Минимум  $G_{ij} D_{ij}$  достигается при:

$$D_{ij}^* = -\operatorname{sign}(G_{ij})$$

Двойственная норма:  $\|G\|_\infty^* = \|G\|_1 = \sum_{ij} |G_{ij}|$

$$W_{k+1} = W_k - \alpha \operatorname{sign}(G_k) = \text{SignGD}$$

## Вывод: Муон как наискорейший спуск в спектральной норме

$$D^* = \operatorname{argmin}_{\|D\|_\sigma \leq 1} \langle G, D \rangle, \quad G = U\Sigma V^\top \text{ (SVD)}$$

## Вывод: Муон как наискорейший спуск в спектральной норме

$$D^* = \operatorname{argmin}_{\|D\|_\sigma \leq 1} \langle G, D \rangle, \quad G = U \Sigma V^\top \text{ (SVD)}$$

Двойственная норма к спектральной — **ядерная**:  $\|G\|_\sigma^* = \|G\|_* = \sum_i \sigma_i(G)$

## Вывод: Муон как наискорейший спуск в спектральной норме

$$D^* = \operatorname{argmin}_{\|D\|_\sigma \leq 1} \langle G, D \rangle, \quad G = U \Sigma V^\top \text{ (SVD)}$$

Двойственная норма к спектральной — **ядерная**:  $\|G\|_\sigma^* = \|G\|_* = \sum_i \sigma_i(G)$

$\operatorname{LMO}_{\|\cdot\|_\sigma}(G)$ : ищем  $\|D\|_\sigma = 1$  (т.е.  $\sigma_{\max}(D) = 1$ ), минимизирующую  $\langle G, D \rangle = \operatorname{tr}(G^\top D)$ .

По неравенству фон Неймана:  $\operatorname{tr}(G^\top D) \geq -\sum_i \sigma_i(G) \sigma_i(D) \geq -\sum_i \sigma_i(G)$

## Вывод: Муон как наискорейший спуск в спектральной норме

$$D^* = \operatorname{argmin}_{\|D\|_\sigma \leq 1} \langle G, D \rangle, \quad G = U \Sigma V^\top \text{ (SVD)}$$

Двойственная норма к спектральной — **ядерная**:  $\|G\|_\sigma^* = \|G\|_* = \sum_i \sigma_i(G)$

$\text{LMO}_{\|\cdot\|_\sigma}(G)$ : ищем  $\|D\|_\sigma = 1$  (т.е.  $\sigma_{\max}(D) = 1$ ), минимизирующую  $\langle G, D \rangle = \operatorname{tr}(G^\top D)$ .

По неравенству фон Неймана:  $\operatorname{tr}(G^\top D) \geq -\sum_i \sigma_i(G) \sigma_i(D) \geq -\sum_i \sigma_i(G)$

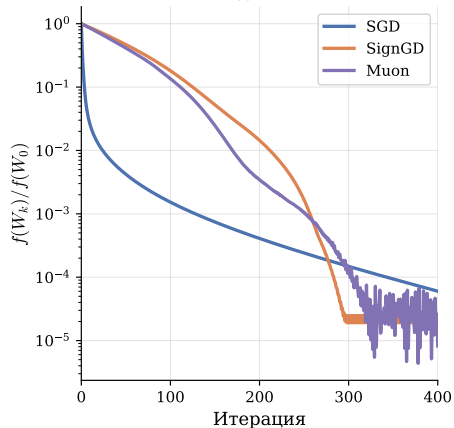
Равенство при  $D^* = -UV^\top$  (все сингулярные числа  $D$  равны 1):

$$W_{k+1} = W_k - \alpha UV^\top = \text{Муон}$$

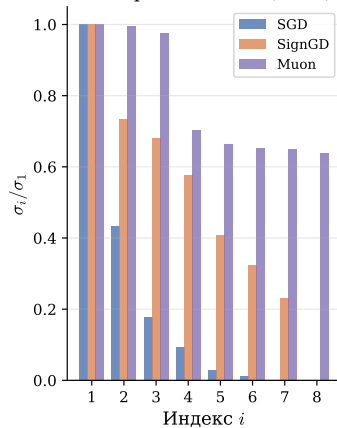
# Три нормы — три оптимизатора: эксперимент

Наискорейший спуск в трёх нормах:  $\|\cdot\|_F$  (SGD),  $\|\cdot\|_\infty$  (SignGD),  $\|\cdot\|_\sigma$  (Muon)

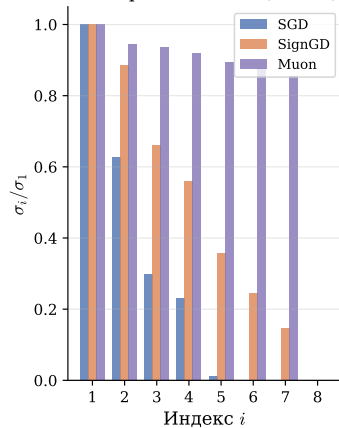
Сходимость



Спектр обновления (iter 0)



Спектр обновления (iter 50)



$f(W) = \frac{1}{2}\|\Sigma W\|_F^2$ ,  $W \in \mathbb{R}^{8 \times 8}$ ,  $\kappa(\Sigma) = 20$ . **Слева:** сходимость трёх методов. **Справа:** спектр обновления  $\Delta W$ . SGD концентрирует энергию на 1–2 сингулярных направлениях, Muon — распределяет равномерно.

# Откуда берётся итерация Newton–Schulz

**Задача:** вычислить  $1/a$  без деления — только сложениями и умножениями.

# Откуда берётся итерация Newton–Schulz

**Задача:** вычислить  $1/a$  без деления — только сложениями и умножениями.

Применим метод Ньютона к  $f(x) = 1/x - a = 0$ :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{1/x_k - a}{-1/x_k^2} = x_k(2 - ax_k).$$



# Откуда берётся итерация Newton–Schulz

**Задача:** вычислить  $1/a$  без деления — только сложениями и умножениями.

Применим метод Ньютона к  $f(x) = 1/x - a = 0$ :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{1/x_k - a}{-1/x_k^2} = x_k(2 - ax_k).$$

Невязка  $u_k = 1 - ax_k$  удовлетворяет  $u_{k+1} = u_k^2$  — **квадратичная сходимость** при  $|u_0| < 1$ .

## Откуда берётся итерация Newton–Schulz

**Задача:** вычислить  $1/a$  без деления — только сложениями и умножениями.

Применим метод Ньютона к  $f(x) = 1/x - a = 0$ :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{1/x_k - a}{-1/x_k^2} = x_k(2 - ax_k).$$

Невязка  $u_k = 1 - ax_k$  удовлетворяет  $u_{k+1} = u_k^2$  — **квадратичная сходимость** при  $|u_0| < 1$ .

Заменяем скаляры на матрицы:  $x_k \rightarrow X_k$ ,  $a \rightarrow A$ :

$$X_{k+1} = X_k(2I - AX_k) \rightarrow A^{-1}$$

Ни одного обращения матрицы — только умножения.

## От обращения матрицы к полярному фактору

Теперь **другая задача**: вычислить  $1/\sqrt{a}$ . Метод Ньютона к  $f(x) = 1/x^2 - a = 0$ :

$$x_{k+1} = x_k - \frac{1/x_k^2 - a}{-2/x_k^3} = \frac{1}{2}x_k(3 - ax_k^2).$$

## От обращения матрицы к полярному фактору

Теперь **другая задача**: вычислить  $1/\sqrt{a}$ . Метод Ньютона к  $f(x) = 1/x^2 - a = 0$ :

$$x_{k+1} = x_k - \frac{1/x_k^2 - a}{-2/x_k^3} = \frac{1}{2}x_k(3 - ax_k^2).$$

Заменяем скаляры на матрицы:  $x_k \rightarrow X_k$ ,  $a \rightarrow X_k^\top X_k$ :

$$X_{k+1} = \frac{1}{2}X_k(3I - X_k^\top X_k)$$

## От обращения матрицы к полярному фактору

Теперь **другая задача**: вычислить  $1/\sqrt{a}$ . Метод Ньютона к  $f(x) = 1/x^2 - a = 0$ :

$$x_{k+1} = x_k - \frac{1/x_k^2 - a}{-2/x_k^3} = \frac{1}{2}x_k(3 - ax_k^2).$$

Заменяем скаляры на матрицы:  $x_k \rightarrow X_k$ ,  $a \rightarrow X_k^\top X_k$ :

$$X_{k+1} = \frac{1}{2}X_k(3I - X_k^\top X_k)$$

**Ключевой трюк**: инициализируем  $X_0 = G/\|G\|_F$ . Итерация сохраняет сингулярные векторы  $U$ ,  $V$  и действует на каждое сингулярное число как  $\sigma \mapsto \frac{1}{2}\sigma(3 - \sigma^2)$ .

## От обращения матрицы к полярному фактору

Теперь **другая задача**: вычислить  $1/\sqrt{a}$ . Метод Ньютона к  $f(x) = 1/x^2 - a = 0$ :

$$x_{k+1} = x_k - \frac{1/x_k^2 - a}{-2/x_k^3} = \frac{1}{2}x_k(3 - ax_k^2).$$

Заменяем скаляры на матрицы:  $x_k \rightarrow X_k$ ,  $a \rightarrow X_k^\top X_k$ :

$$X_{k+1} = \frac{1}{2}X_k(3I - X_k^\top X_k)$$

**Ключевой трюк**: инициализируем  $X_0 = G/\|G\|_F$ . Итерация сохраняет сингулярные векторы  $U$ ,  $V$  и действует на каждое сингулярное число как  $\sigma \mapsto \frac{1}{2}\sigma(3 - \sigma^2)$ .

Неподвижная точка:  $\sigma = 1 \Rightarrow \sigma_i(X_k) \rightarrow 1 \Rightarrow X_k \rightarrow UV^\top$ . Множитель  $G$  «вшит» в  $X_0$ ; итерация лишь выравнивает спектр, не трогая направления.

---

**Algorithm 1** Proposed optimizer

---

**Require:** Learning rate  $\eta$ , momentum  $\mu$ , weight decay  $\lambda$

- 1: Initialize  $B_0 \leftarrow 0$
  - 2: **for**  $t = 1, \dots$  **do**
  - 3:   Compute gradient  $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
  - 4:    $B_t \leftarrow \mu B_{t-1} + G_t$
  - 5:    $G_t \leftarrow \mu B_t + G_t$
  - 6:    $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
  - 7:   Update parameters  $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda\theta_{t-1})$
  - 8: **end for**
  - 9: **return**  $\theta_t$
- 

**Newton-Schulz** вместо SVD для вычисления  $UV^{\top}$ :

$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами  $a=3.4445$ ,  $b=-4.7750$ ,  
 $c=2.0315$ .

- **5 итераций** достаточно на практике

---

**Algorithm 1** Proposed optimizer

---

**Require:** Learning rate  $\eta$ , momentum  $\mu$ , weight decay  $\lambda$

- 1: Initialize  $B_0 \leftarrow 0$
  - 2: **for**  $t = 1, \dots$  **do**
  - 3:   Compute gradient  $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
  - 4:    $B_t \leftarrow \mu B_{t-1} + G_t$
  - 5:    $G_t \leftarrow \mu B_t + G_t$
  - 6:    $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
  - 7:   Update parameters  $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda \theta_{t-1})$
  - 8: **end for**
  - 9: **return**  $\theta_t$
- 

**Newton-Schulz** вместо SVD для вычисления  $UV^{\top}$ :

$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами  $a=3.4445$ ,  $b=-4.7750$ ,  
 $c=2.0315$ .

- **5 итераций** достаточно на практике
- Только матричные умножения — эффективно на GPU



---

**Algorithm 1** Proposed optimizer

---

**Require:** Learning rate  $\eta$ , momentum  $\mu$ , weight decay  $\lambda$

- 1: Initialize  $B_0 \leftarrow 0$
  - 2: **for**  $t = 1, \dots$  **do**
  - 3:   Compute gradient  $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
  - 4:    $B_t \leftarrow \mu B_{t-1} + G_t$
  - 5:    $G_t \leftarrow \mu B_t + G_t$
  - 6:    $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
  - 7:   Update parameters  $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda \theta_{t-1})$
  - 8: **end for**
  - 9: **return**  $\theta_t$
- 

**Newton-Schulz** вместо SVD для вычисления  $UV^{\top}$ :

$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами  $a=3.4445$ ,  $b=-4.7750$ ,  
 $c=2.0315$ .

- **5 итераций** достаточно на практике
- Только матричные умножения — эффективно на GPU
- Кубическая сходимость к полярному фактору

## Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

## Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.

## Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.

## Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.
- **Muon**<sup>15</sup>:  $p(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$  — **эвристический** градиентный поиск. Не сходится к  $UV^\top$  (ошибка  $\approx 0.3$ ), но на практике достаточно для обучения.

---

<sup>15</sup>  К. Jordan, блог-пост (2024)

## Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.
- **Muon**<sup>15</sup>:  $p(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$  — **эвристический** градиентный поиск. Не сходится к  $UV^\top$  (ошибка  $\approx 0.3$ ), но на практике достаточно для обучения.

---

<sup>15</sup>  К. Jordan, блог-пост (2024)

# Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.
- **Muon**<sup>15</sup>:  $p(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$  — **эвристический** градиентный поиск. Не сходится к  $UV^\top$  (ошибка  $\approx 0.3$ ), но на практике достаточно для обучения.
- **Polar Express**<sup>16</sup>: **адаптивный** полином на каждой итерации, жадный минимакс  $\rightarrow$  доказанная оптимальность.

---

<sup>15</sup>  К. Jordan, блог-пост (2024)

<sup>16</sup>  Amsel et al. (2025)

# Откуда берутся коэффициенты?

Полином  $p(\sigma) \rightarrow 1$  действует на сингулярные числа. Задача — **минимаксная аппроксимация единицы** нечётным полиномом:

$$p^* = \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell, u]} |1 - p(x)|$$

- **Классический NS** (Schulz, 1933):  $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ . Нечётный кубический полином имеет вид  $p(x) = ax - bx^3$ . Условия  $p(1) = 1$  и  $p'(1) = 0$  (неподвижная точка с нулевой производной для квадратичной сходимости) дают систему  $a - b = 1$ ,  $a - 3b = 0$ , откуда  $a = 3/2$ ,  $b = 1/2$ . Медленный начальный этап.
- **Muon**<sup>15</sup>:  $p(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$  — **эвристический** градиентный поиск. Не сходится к  $UV^\top$  (ошибка  $\approx 0.3$ ), но на практике достаточно для обучения.
- **Polar Express**<sup>16</sup>: **адаптивный** полином на каждой итерации, жадный минимакс  $\rightarrow$  доказанная оптимальность.
- **CANS**<sup>17</sup>: оптимальные полиномы через **равноосцилляцию Чебышёва**. Явные формулы для степени 3, алгоритм Ремеза для  $d > 3$ .

---

<sup>15</sup>  К. Jordan, блог-пост (2024)

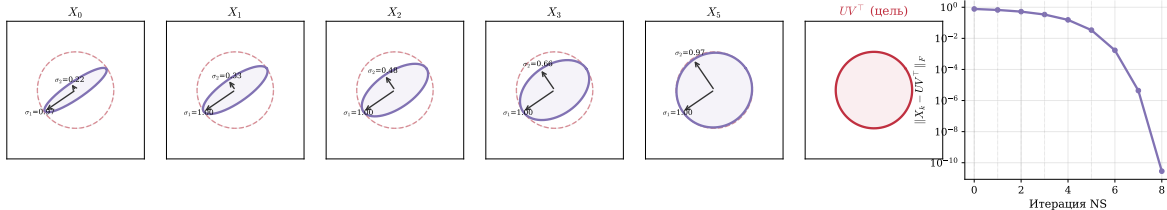
<sup>16</sup>  Amsel et al. (2025)

<sup>17</sup>  Grishina et al. (2025)



# Newton-Schulz: эллипс $\rightarrow$ окружность

Newton-Schulz: эллипс  $\rightarrow$  окружность (ортогонализация)

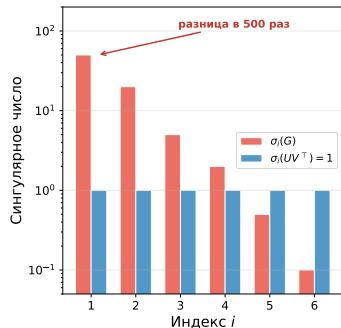


Пусть  $G = U\Sigma V^\top$ ,  $X_0 = G/\|G\|_F$ . Итерация  $X_{k+1} = \frac{1}{2}X_k(3I - X_k^\top X_k)$  сохраняет сингулярные векторы  $U$ ,  $V$  и применяет к каждому сингулярному числу отображение  $\sigma \mapsto \frac{1}{2}\sigma(3 - \sigma^2)$ . Неподвижная точка:  $\sigma = 1$ . Сходимость:  $\sigma_i(X_k) \rightarrow 1 \Rightarrow X_k \rightarrow UV^\top$ .

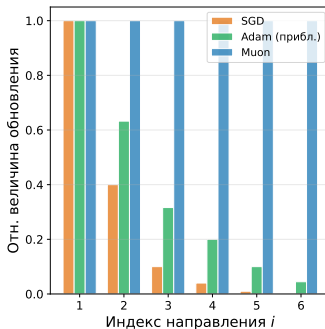
## Результаты и эксперименты

# Муон: геометрия ортогонализации

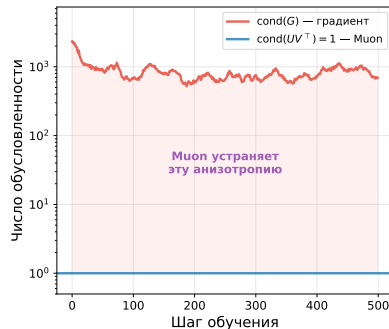
Сингулярные числа  
градиента и Муон



Энергия обновления  
по направлениям



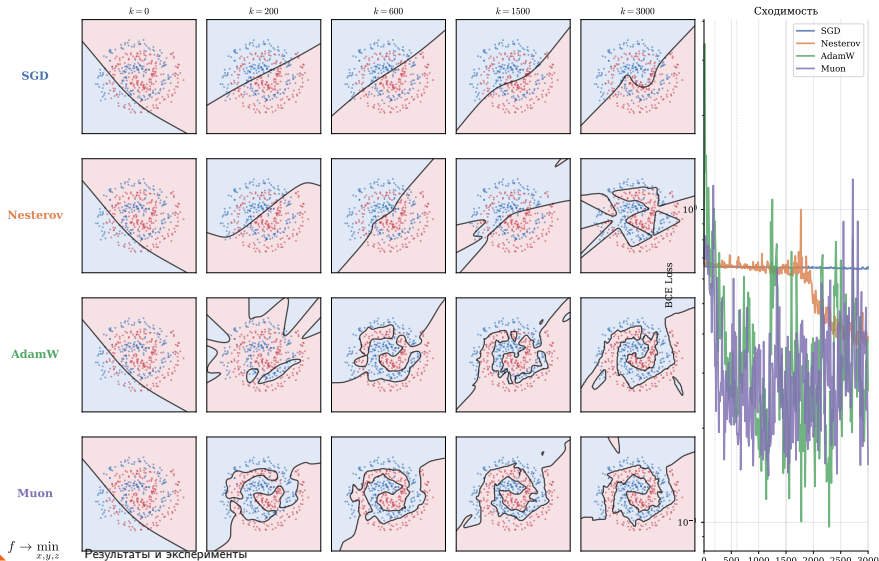
Число обусловленности



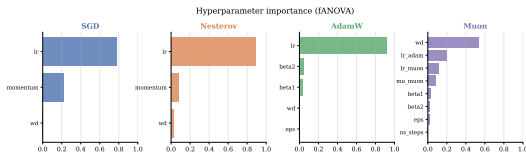
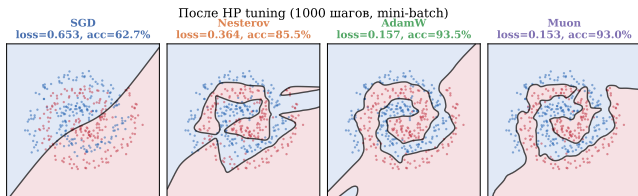
**Вывод:** SGD концентрирует обновление на доминирующих сингулярных направлениях (разница  $500\times$ ). Муон нормализует все направления к единице — **равномерный прогресс** во всех направлениях пространства параметров.

# Эксперимент: классификация спиралей

Классификация спиралей (mini-batch): SGD, Nesterov, AdamW, Muon



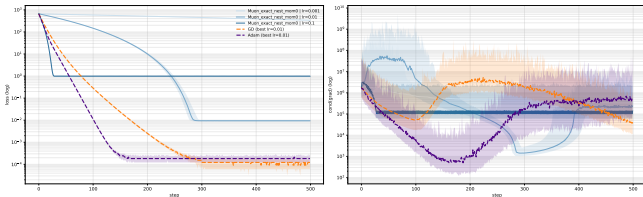
# Классификация спиралей: после HP tuning (Optuna)



500 trials  $\times$  1000 steps. SGD/Nesterov: **lr доминирует** (>70%). Muon: **weight decay** и **lr равнозначны** — ортогонализация снижает чувствительность к шагу.

 Интерактивная визуализация и детали

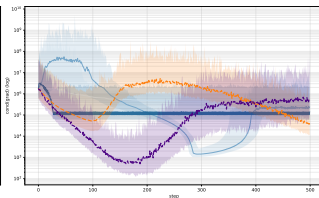
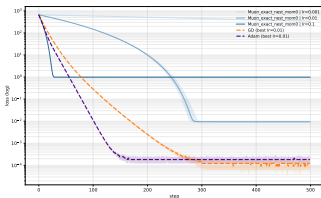
# Муон на квадратичных задачах <sup>18</sup>



Простейшая задача:  $L(W) = \frac{1}{2}\|W\|_F^2$ , точная ортогонализация (без Newton-Schulz), без момента.

- Обновление Муон:  $W_{k+1} = W_k - \alpha UV^\top$ , где  $W_k = U\Sigma V^\top$ . Каждое сингулярное число обновляется независимо:  
 $\sigma_{k+1} = \sigma_k - \alpha \text{sign}(\sigma_k)$

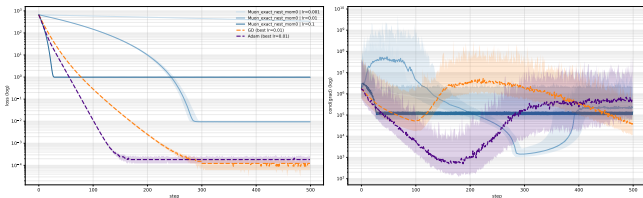
# Муон на квадратичных задачах <sup>18</sup>



Простейшая задача:  $L(W) = \frac{1}{2}\|W\|_F^2$ , точная ортогонализация (без Newton-Schulz), без момента.

- Обновление Муон:  $W_{k+1} = W_k - \alpha UV^\top$ , где  $W_k = U\Sigma V^\top$ . Каждое сингулярное число обновляется независимо:  
 $\sigma_{k+1} = \sigma_k - \alpha \text{sign}(\sigma_k)$
- Шаг **фиксирован** по модулю ( $\alpha$ ) и не зависит от  $|\sigma_k|$  — аналог предельного цикла Adam, но для каждого сингулярного числа

# Муон на квадратичных задачах <sup>18</sup>



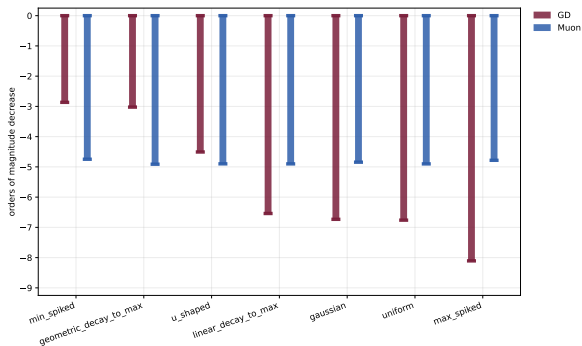
Простейшая задача:  $L(W) = \frac{1}{2}\|W\|_F^2$ , точная ортогонализация (без Newton-Schulz), без момента.

- Обновление Муон:  $W_{k+1} = W_k - \alpha UV^\top$ , где  $W_k = U\Sigma V^\top$ . Каждое сингулярное число обновляется независимо:  
 $\sigma_{k+1} = \sigma_k - \alpha \text{sign}(\sigma_k)$
- Шаг **фиксирован** по модулю ( $\alpha$ ) и не зависит от  $|\sigma_k|$  — аналог предельного цикла Adam, но для каждого сингулярного числа
- Итог: **2-цикл** около нуля, loss не опускается ниже  $\Theta(\alpha^2)$ . Скорость  $O(1/\sqrt{\epsilon})$  шагов против  $O(\log(1/\epsilon))$  у GD

<sup>18</sup>  Gonon, Muşat, Boumal (2026). Insights on Muon from Simple Quadratics.



## Муон: стабильность по спектру задачи <sup>19</sup>

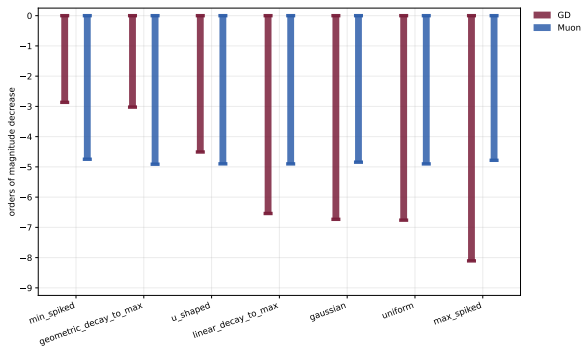


$$L(W) = \frac{1}{2} \langle W, AW \rangle + \langle B, W \rangle + c, \quad A = QSQ^\top,$$

$n = 100$ .

7 спектров  $S$ , все с  $(s_{\min}, s_{\max}) = (10^{-3}, 10)$ ,  $\kappa = 10^4$ ,  
 $T = 500$ .

# Муон: стабильность по спектру задачи <sup>19</sup>

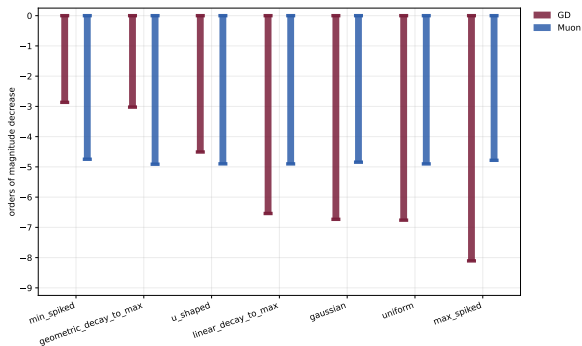


$$L(W) = \frac{1}{2} \langle W, AW \rangle + \langle B, W \rangle + c, \quad A = QSQ^\top, \\ n = 100.$$

7 спектров  $S$ , все с  $(s_{\min}, s_{\max}) = (10^{-3}, 10)$ ,  $\kappa = 10^4$ ,  $T = 500$ .

**Почему Муон стабилен:** обновление  $-UV^\top$  имеет все сингулярные числа  $= 1 \rightarrow$  одинаковый прогресс по **всем** направлениям, независимо от кривизны.

# Муон: стабильность по спектру задачи <sup>19</sup>



$$L(W) = \frac{1}{2} \langle W, AW \rangle + \langle B, W \rangle + c, \quad A = QSQ^\top, \\ n = 100.$$

7 спектров  $S$ , все с  $(s_{\min}, s_{\max}) = (10^{-3}, 10)$ ,  $\kappa = 10^4$ ,  $T = 500$ .

**Почему Муон стабилен:** обновление  $-UV^\top$  имеет все сингулярные числа  $= 1 \rightarrow$  одинаковый прогресс по **всем** направлениям, независимо от кривизны.

**Почему GD нестабилен:** обновление  $\propto \nabla L = AW \rightarrow$  прогресс пропорционален собственному числу  $s_i$ . Если большинство  $s_i \approx s_{\min}$  (min\_spiked) — GD медленный. Если  $s_i \approx s_{\max}$  (max\_spiked) — быстрый. При одном  $\kappa$ , но разной форме спектра, ранжирование Muon vs GD **переворачивается**.

## Мagma: случайное маскирование обновлений <sup>20</sup>

**Идея:** случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

**SkipUpdate:** на каждом шаге блокируем обновление блока параметров с вероятностью  $1 - p$ , масштабируя оставшиеся на  $1/p$ .

## Мagma: случайное маскирование обновлений <sup>20</sup>

**Идея:** случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

**SkipUpdate:** на каждом шаге блокируем обновление блока параметров с вероятностью  $1 - p$ , масштабируя оставшиеся на  $1/p$ .

**Мagma** (Momentum-Aligned Gradient Masking): модулирует маскирование через выравнивание с моментумом:

$$\tilde{s}_t^{(b)} = \text{sigmoid} \left( \frac{\cos(\mu_t^{(b)}, g_t^{(b)})}{\tau} \right)$$

---

<sup>20</sup>  Joo, T. et al. (2026). On Surprising Effectiveness of Masking Updates in Adaptive Optimizers.

## Magma: случайное маскирование обновлений <sup>20</sup>

**Идея:** случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

**SkipUpdate:** на каждом шаге блокируем обновление блока параметров с вероятностью  $1 - p$ , масштабируя оставшиеся на  $1/p$ .

**Magma** (Momentum-Aligned Gradient Masking): модулирует маскирование через выравнивание с моментумом:

$$\tilde{s}_t^{(b)} = \text{sigmoid} \left( \frac{\cos(\mu_t^{(b)}, g_t^{(b)})}{\tau} \right)$$

**Теоретический инсайт:** маскирование индуцирует геометрическую регуляризацию, зависящую от кривизны:

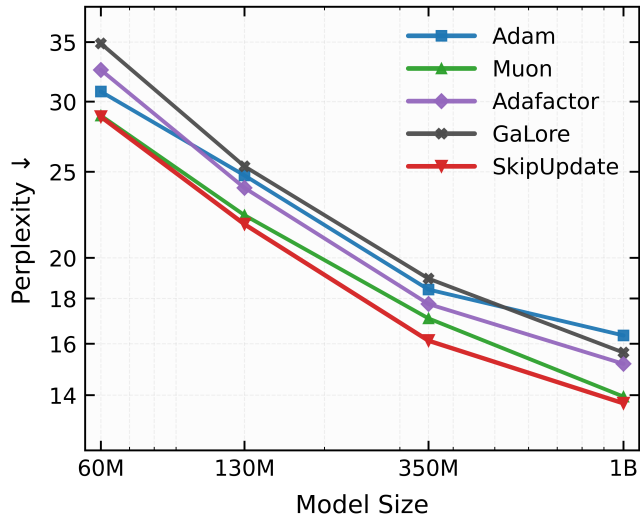
$$\mathcal{R}_t^{(b)} = \frac{1-p}{2p} \cdot (\Delta_t^{(b)})^\top H_{bb}(\theta_t) \Delta_t^{(b)}$$

Штрафует обновления вдоль направлений с большой кривизной — оптимизация смещается к более плоским минимумам.

---

<sup>20</sup>  Joo, T. et al. (2026). On Surprising Effectiveness of Masking Updates in Adaptive Optimizers.

## Магма: масштабирование до 1B параметров <sup>21</sup>



Магма превосходит Adam, Muon, Adafactor, GaLore на всех масштабах. На 1B: perplexity **13.8** vs 16.3 (Adam).

<sup>21</sup>  Joo et al. (2026)

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.



## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).
- Ключевой инсайт: ортогонализация  $(UV^\top)$  устраняет зависимость от числа обусловленности — **равномерный прогресс** по всем сингулярным направлениям.

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).
- Ключевой инсайт: ортогонализация  $(UV^\top)$  устраняет зависимость от числа обусловленности — **равномерный прогресс** по всем сингулярным направлениям.
- Newton-Schulz позволяет эффективно вычислять полярный фактор: 5 итераций матричных умножений вместо полного SVD.

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).
- Ключевой инсайт: ортогонализация  $(UV^\top)$  устраняет зависимость от числа обусловленности — **равномерный прогресс** по всем сингулярным направлениям.
- Newton-Schulz позволяет эффективно вычислять полярный фактор: 5 итераций матричных умножений вместо полного SVD.

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).
- Ключевой инсайт: ортогонализация ( $UV^\top$ ) устраняет зависимость от числа обусловленности — **равномерный прогресс** по всем сингулярным направлениям.
- Newton-Schulz позволяет эффективно вычислять полярный фактор: 5 итераций матричных умножений вместо полного SVD.
- **На практике:** Muon — лучший по NanoGPT speedrun; используется в моделях масштаба 1T (Kimi K2).

## Итоги лекции

- **Muon** — метод ортогонализации градиента, делающий **наискорейший спуск в спектральной норме** для матричных параметров.
- Shampoo приближает полноматричный предобусловитель AdaGrad через произведения Кронекера; Muon — его предельный случай (один градиент).
- Ключевой инсайт: ортогонализация ( $UV^\top$ ) устраняет зависимость от числа обусловленности — **равномерный прогресс** по всем сингулярным направлениям.
- Newton-Schulz позволяет эффективно вычислять полярный фактор: 5 итераций матричных умножений вместо полного SVD.
- **На практике:** Muon — лучший по NanoGPT speedrun; используется в моделях масштаба 1T (Kimi K2).
- **Magma** — регуляризация через маскирование, смещающая обучение к плоским минимумам.

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"



## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
-  Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
-  Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)
-  Amsel et al. "The Polar Express: Optimal Matrix Sign Methods" (2025)

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
-  Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)
-  Amsel et al. "The Polar Express: Optimal Matrix Sign Methods" (2025)
-  Grishina et al. "Accelerating Newton-Schulz via Chebyshev-type Polynomials" (2025)

## Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
-  Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)
-  Amsel et al. "The Polar Express: Optimal Matrix Sign Methods" (2025)
-  Grishina et al. "Accelerating Newton-Schulz via Chebyshev-type Polynomials" (2025)
-  Д. Ковалёв "Understanding Gradient Orthogonalization" (теория Muon)

## Приложение

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .

**Замечания:**

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .

**Замечания:**



**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .
3. Вычислить предобусловители  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$  (обратный матричный корень).

**Замечания:**

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .
3. Вычислить предобусловители  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$  (обратный матричный корень).
4. Обновление:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .
3. Вычислить предобусловители  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$  (обратный матричный корень).
4. Обновление:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Учитывает информацию о кривизне эффективнее методов первого порядка.

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .
3. Вычислить предобусловители  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$  (обратный матричный корень).
4. Обновление:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Учитывает информацию о кривизне эффективнее методов первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее по числу шагов.

**Shampoo** = **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks — метод, вдохновлённый оптимизацией второго порядка.

**Основная идея:** Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов  $W \in \mathbb{R}^{m \times n}$ :

1. Вычислить градиент  $G_k$ .
2. Обновить статистику  $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$ .
3. Вычислить предобусловители  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$  (обратный матричный корень).
4. Обновление:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Учитывает информацию о кривизне эффективнее методов первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее по числу шагов.
- Требуется аккуратной реализации (эффективное вычисление обратных матричных корней).

<sup>22</sup>  Gupta, Koren, Singer (2018). Shampoo: Preconditioned Stochastic Tensor Optimization. ICML.

## Муон как упрощение Shampoo <sup>23</sup>

$$W_{t+1} = W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}}$$

## Муон как упрощение Shampoo <sup>23</sup>

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \end{aligned}$$

## Муон как упрощение Shampoo <sup>23</sup>

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\ &= W_t - \eta (U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \end{aligned}$$



## Муон как упрощение Shampoo <sup>23</sup>

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\ &= W_t - \eta (U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \\ &= W_t - \eta U S^{-1/2} S S^{-1/2} V^\top \end{aligned}$$

## Муон как упрощение Shampoo <sup>23</sup>

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\ &= W_t - \eta (U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \\ &= W_t - \eta U S^{-1/2} S S^{-1/2} V^\top \\ &= W_t - \eta U V^\top \end{aligned}$$

---

<sup>23</sup>  J. Bernstein “Deriving Muon”

## Муон как упрощение Shampoo <sup>23</sup>

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\ &= W_t - \eta (U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \\ &= W_t - \eta U S^{-1/2} S S^{-1/2} V^\top \\ &= W_t - \eta U V^\top \end{aligned}$$

**Ключевой инсайт:** Shampoo с «идеальной» статистикой (один градиент вместо накопленных) — это просто ортогонализация градиента. Все сингулярные числа обновления равны 1.

---

<sup>23</sup>  J. Bernstein “Deriving Muon”